

Day 1 — Thinking in Algorithms

Problem solving & computational thinking

Algorithm constructs

Big O & complexity analysis

ADTs as OO design

Before we build a single data structure, we learn how to **think** about a problem and how to **measure** a solution. Everything in the next 11 days depends on today.

1. Why this module matters

We need to answer one question: **"Is there any duplicate roll number in this list?"**

Two students write two correct Java programs. Both give the right answer. Here is how long each one took on the same laptop:

Students (n)	Program A — compare every pair	Program B — use a HashSet
10,000	17 ms	2 ms
40,000	188 ms	4 ms
1,60,000	2,467 ms	38 ms
1 crore (estimated)	about 2.7 hours	under 3 seconds

Same computer, same Java, same data. **Only the algorithm changed.**

- Program A does about $n^2/2$ comparisons. When n becomes $4\times$, time becomes about **16 $\times$** .
- Program B does about n operations. When n becomes $4\times$, time becomes about **4 $\times$** .
- Choosing the right **data structure** (here, a hash table) is what made Program B fast.

TRY IT Run it yourself

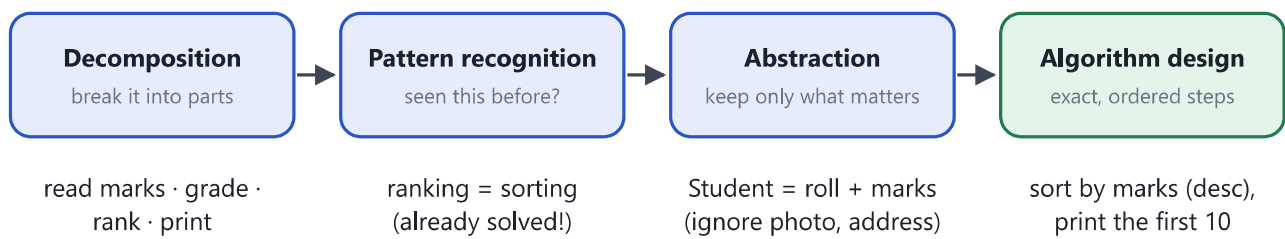
Open `DuplicateCheck.java` from today's code folder and run it. Your numbers will be different — the **pattern** will be the same.

This module is about exactly this skill: **choosing and building the right data structure and algorithm, and proving why it is better.**

2. Problem solving & computational thinking

DEFINITION Computational thinking

A way of solving problems so that the solution can be carried out by a computer: break the problem down, spot patterns, ignore unimportant details, and write exact steps.

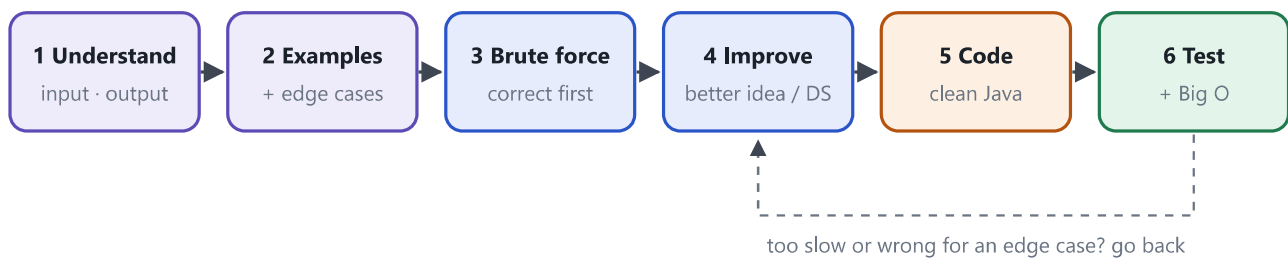


Example problem: "Publish the top-10 rank list of 300 students"

The four pillars of computational thinking, applied to one problem

- **Decomposition** — a big problem is hard; many small problems are easy. "Build a result system" becomes: read marks → calculate grade → rank → print.
- **Pattern recognition** — most "new" problems are old problems in disguise. Ranking is sorting. "Undo" is a stack. "Serve customers in order" is a queue.
- **Abstraction** — keep only the details that affect the answer. For ranking, a student is just (*roll number, marks*).
- **Algorithm design** — write the steps so exactly that a computer (or a new student) can follow them without guessing.

How we will solve every problem in this module



Our standard routine — we will use it on every problem, every day

Worked example: find the second largest number

Step 1 — Understand. Input: an `int[]`. Output: the second largest **distinct** value (or "none").

Step 2 — Examples and edge cases. Good programmers find the tricky inputs *before* coding.

Input	Expected	Why it is tricky
{12, 35, 1, 10, 34, 1}	34	normal case
{10, 10, 10}	none	all values equal
{7, 7, 3}	3	the maximum is repeated
{-5, -2, -9}	-5	negatives: starting with 0 would be a bug
{42}	none	fewer than 2 elements

Step 3 — Brute force. Sort, then walk back from the end until the value changes. Correct, but sorting costs $O(n \log n)$.

Step 4 — Improve. One pass. Remember the **best** and the **second best** seen so far.

PSEUDOCODE

```

ALGORITHM SecondLargest(A[0..n-1])
  IF n < 2 THEN RETURN none
  first ← -∞ ; second ← -∞
  FOR each x in A DO

```

```

IF x > first THEN
    second ← first      // old maximum moves down
    first ← x
ELSE IF x < first AND x > second THEN
    second ← x
RETURN (second =  $-\infty$ ) ? none : second

```

Step 5 — Code.

JAVA SecondLargest.java

```

static Integer onePass(int[] a) {
    if (a.length < 2) return null;
    long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
    for (int x : a) {
        if (x > first) {
            second = first;
            first = x;
        } else if (x < first && x > second) {
            second = x;
        }
    }
    return second == Long.MIN_VALUE ? null : (int) second;
}

```

Step 6 — Test and analyse. Run all five inputs from the table. One loop over n elements → **$O(n)$ time**, two variables → **$O(1)$ extra space**.

QUICK CHECK Why long and not int for first and second ?

Think about the array {Integer.MIN_VALUE, 5} .

Show answer

3. What is an algorithm?

DEFINITION Algorithm

A finite sequence of well-defined steps that takes some input and produces the required output.

Every algorithm must have these five properties:

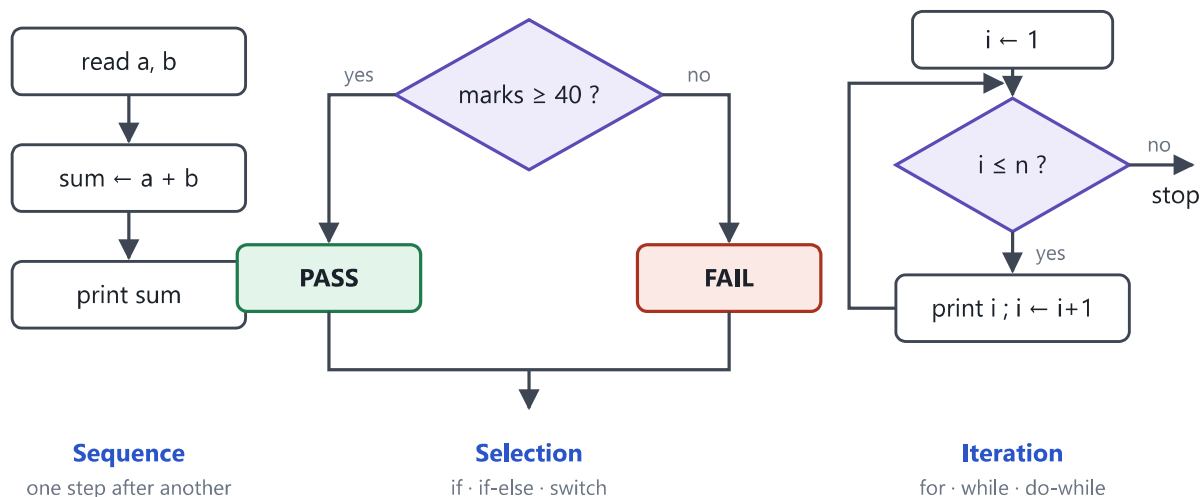
1. **Input** — zero or more values given to it.
2. **Output** — at least one result.
3. **Definiteness** — every step is clear. "Add a few numbers" is not definite; "add `A[i]` to `sum`" is.
4. **Finiteness** — it must stop after a finite number of steps.
5. **Effectiveness** — every step is basic enough to actually be carried out.

Algorithm	Program
An idea — the steps	Those steps written in a language (Java)
Written in English, pseudocode or a flowchart	Must follow Java syntax exactly
Independent of language and machine	Runs on a real JVM
We analyse it (Big O)	We test it (run it)

We will write algorithms in **pseudocode** first, then in Java. Our pseudocode rules: `←` means assignment, blocks are shown by indentation, and keywords are in CAPITALS (`IF` , `FOR` , `WHILE` , `RETURN`).

4. Algorithm constructs

Every algorithm — from finding the maximum to Google Maps' route finder — is built from just **three** building blocks.



The three constructs. Recursion — a method calling itself — is a fourth way to repeat (Day 4).

Construct	Pseudocode	Java
Sequence	statements one below another	statements ending with <code>;</code>
Selection	<code>IF ... THEN ... ELSE</code>	<code>if</code> , <code>if-else</code> , <code>switch</code> , <code>? :</code>
Iteration	<code>FOR</code> , <code>WHILE</code> , <code>REPEAT ... UNTIL</code>	<code>for</code> , enhanced <code>for</code> , <code>while</code> , <code>do-while</code>
Recursion (Day 4)	<code>CALL</code> itself on a smaller input	a method that calls itself

Example — linear search uses all three:

PSEUDOCODE

```

ALGORITHM LinearSearch(A[0..n-1], key)
  FOR i ← 0 TO n-1 DO                // iteration
    IF A[i] = key THEN RETURN i         // selection
  RETURN -1                             // sequence: runs after the loop

```

JAVA LinearSearchCount.java

```

static int linearSearch(int[] a, int key) {
  for (int i = 0; i < a.length; i++) {
    if (a[i] == key) return i;           // found
  }
}

```

```

    }
    return -1;           // not found
}

```

5. Analysing algorithms — why not just use a stopwatch?

We could time a program with `System.nanoTime()`. But a stopwatch tells us about **one run on one machine**, not about the algorithm:

- A faster laptop gives a smaller number for the same algorithm.
- The JVM's JIT compiler makes the same code faster after it "warms up".
- Other programs running at the same time disturb the measurement.
- Most importantly, it tells us nothing about what happens when the input becomes **10× or 1000× bigger**.

So instead we **count the basic operations as a function of the input size n** . We call this count **$T(n)$** .

JAVA

```

static long sum(int[] a) {
    long s = 0;           // 1
    for (int i = 0; i < a.length; i++) // i=0 once, compare n+1 times, i++ n times
        s += a[i];        // n times
    return s;             // 1
}

```

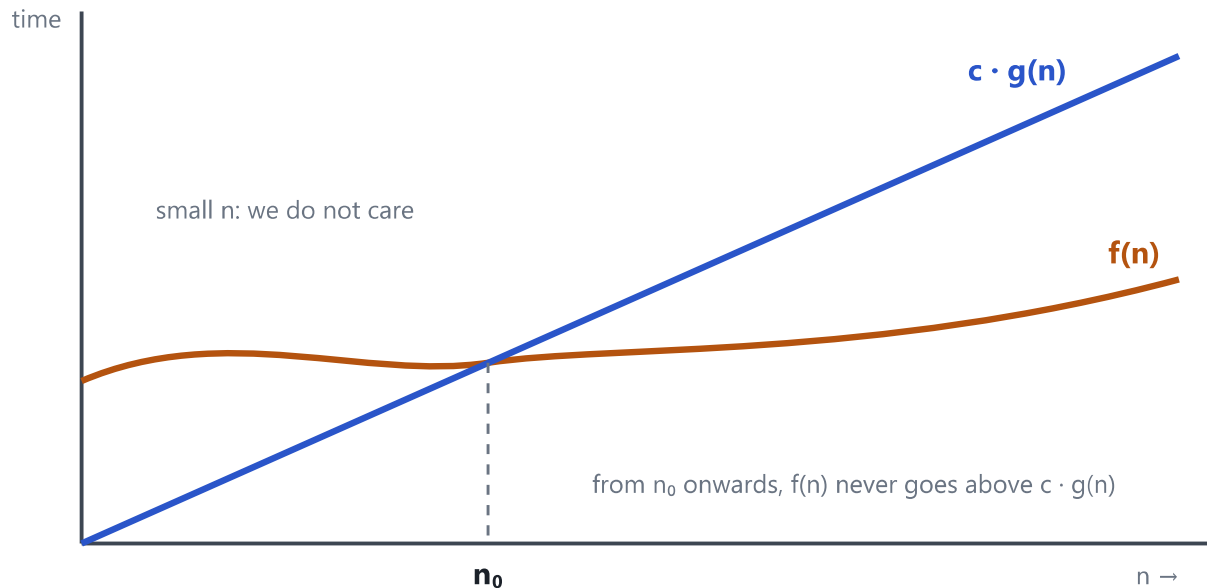
Adding it up: $T(n) = 1 + 1 + (n + 1) + n + n + 1 = 3n + 4$.

Exact counting is tiring, and the exact constants depend on the compiler anyway. What we care about is the **shape of the growth**: when n doubles, $3n + 4$ roughly doubles. That shape is what Big O captures.

6. Asymptotic notation — Big O, Ω and Θ

DEFINITION Big O (upper bound)

$f(n) = O(g(n))$ if there are positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for every $n \geq n_0$. In words: *for large inputs, f grows no faster than g .*



Big O ignores small inputs and constant factors — it describes the long-run growth

Example: $T(n) = 3n + 4$. Choose $c = 4$ and $n_0 = 4$. For every $n \geq 4$, $3n + 4 \leq 4n$. So $T(n) = O(n)$.

Notation	Meaning	Think of it as
$O(g)$ — Big O	grows no faster than g	upper bound, "at most" ($\leq$)
$\Omega(g)$ — Big Omega	grows no slower than g	lower bound, "at least" ($\geq$)
$\Theta(g)$ — Big Theta	grows exactly like g	tight bound ($=$) — both O and Ω

In practice (and in interviews) we usually quote the **Big O of the worst case**: "binary search is $O(\log n)$ ".

The rules that make Big O quick

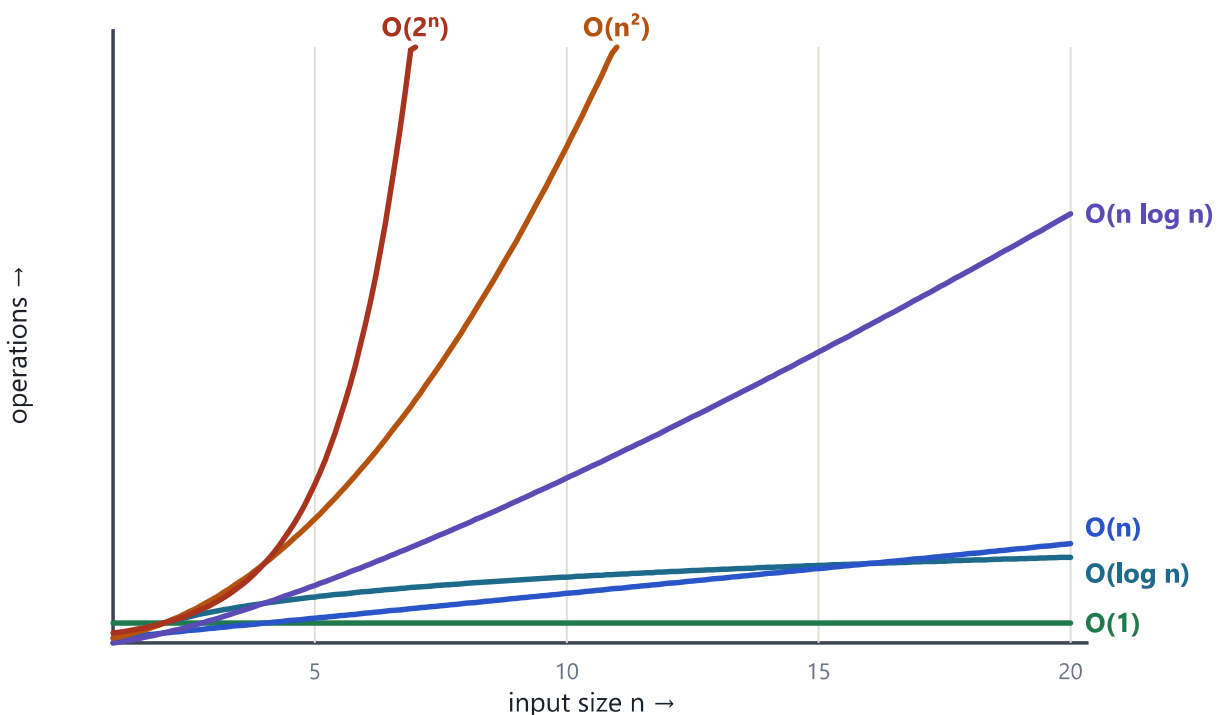
1. **Drop constant factors** — $5n \rightarrow O(n)$, $n^2/2 \rightarrow O(n^2)$.
2. **Drop lower-order terms** — $n^2 + 100n + 5000 \rightarrow O(n^2)$.
3. **Sequential blocks add** — a loop of n , then a loop of $m \rightarrow O(n + m)$.
4. **Nested blocks multiply** — a loop of n inside a loop of $n \rightarrow O(n \times n) = O(n^2)$.
5. **Different inputs, different letters** — two separate arrays of sizes n and m give $O(n \times m)$, **not** $O(n^2)$.

Why is dropping terms fair? Look at $3n^2 + 5n + 100$ when $n = 1000$:

Term	Value	Share of the total
$3n^2$	30,00,000	99.8%
$5n$	5,000	0.17%
100	100	0.003%

For large n , the biggest term is the only one that matters.

7. The common complexity classes



How the number of operations grows with n . Anything above $O(n \log n)$ gets expensive quickly. ($O(1)$ and $O(\log n)$ are drawn 4× taller so they are visible.)

Big O	Name	Typical example	In Java
$O(1)$	constant	read <code>a[i]</code> , push onto a stack	<code>ArrayList.get(i)</code> , <code>HashMap.get</code> (average)
$O(\log n)$	logarithmic	binary search; halving each step	<code>Arrays.binarySearch</code> , <code>TreeMap.get</code>
$O(n)$	linear	one loop over the data	<code>ArrayList.contains</code> , <code>LinkedList.get(i)</code>
$O(n \log n)$	linearithmic	good sorting (merge sort, heap sort)	<code>Arrays.sort</code> , <code>Collections.sort</code>
$O(n^2)$	quadratic	two nested loops over the data	bubble sort, comparing every pair
$O(2^n)$	exponential	trying every subset; naive recursive Fibonacci	— avoid for large n
$O(n!)$	factorial	trying every ordering	— only for tiny n

What those functions mean in numbers:

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
10	3.3	33	100	1,024
1,000	10	10,000	10,00,000	a 302-digit number
10,00,000	20	2 crore	10^{12}	impossible

TIP Rule of thumb for Java

A modern machine does roughly **10^8 simple operations per second**. So for $n = 10^6$: $O(n \log n) \approx 2 \times 10^7 \rightarrow$ a fraction of a second. $O(n^2) = 10^{12} \rightarrow$ **hours**. Before coding, put your n into your Big O and see if the answer is sensible.

8. Analysing loops — the patterns

Count **how many times the innermost statement runs**. Almost every loop you write is one of these patterns.

JAVA

```
// Pattern 1 - single loop:  $O(n)$ 
```

```
for (int i = 0; i < n; i++) count++;
```

```
// Pattern 2 - nested, independent:  $n \times n = O(n^2)$ 
```

```
for (int i = 0; i < n; i++)
```

```
    for (int j = 0; j < n; j++) count++;
```

```
// Pattern 3 - nested, inner depends on outer:  $0 + 1 + \dots + (n-1) = n(n-1)/2 =$ 
```

```
for (int i = 0; i < n; i++)
```

```
    for (int j = 0; j < i; j++) count++;
```

```
// Pattern 4 - halving (or doubling):  $O(\log n)$ 
```

```
for (int i = n; i > 1; i /= 2) count++;
```

```
// Pattern 5 - linear outer  $\times$  logarithmic inner:  $O(n \log n)$ 
```

```
for (int i = 0; i < n; i++)
```

```
    for (int j = 1; j < n; j *= 2) count++;
```

```
// Pattern 6 - two loops one after the other:  $O(n) + O(n) = O(n)$ 
```

```
for (int i = 0; i < n; i++) count++;
```

```
for (int i = 0; i < n; i++) count++;
```

```
// Pattern 7 - loop until  $i*i > n$ :  $O(\sqrt{n})$ 
```

```
for (int i = 1; i * i ≤ n; i++) count++;
```

```
// Pattern 8 - fixed number of iterations:  $O(1)$ , no matter how big n is
```

```
for (int i = 0; i < 100; i++) count++;
```

Why is halving $O(\log n)$? Starting from n , after k halvings we have $n / 2^k$. The loop stops when $n / 2^k = 1$, which means $k = \log_2 n$. For $n = 10,00,000$ that is only **20** steps.

Output of `LoopComplexity.java` — watch what happens when n doubles:

n	single $O(n)$	nested $O(n^2)$	halving $O(\log n)$	$n \log n$
1,000	1,000	10,00,000	9	10,000
2,000	2,000	40,00,000	10	22,000
4,000	4,000	1,60,00,000	11	48,000
8,000	8,000	6,40,00,000	12	1,04,000

- $O(n)$: n doubles $\rightarrow$ count **doubles**.
- $O(n^2)$: n doubles $\rightarrow$ count becomes **4x**.
- $O(\log n)$: n doubles $\rightarrow$ count increases by just **1**.

QUICK CHECK What is the Big O? (1)

JAVA

```
for (int i = 1; i ≤ n; i *= 3) System.out.println(i);
```

Show answer

QUICK CHECK What is the Big O? (2)

JAVA

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) total += a[i] * b[j];
```

Show answer

QUICK CHECK What is the Big O? (3)

JAVA

```
for (int i = 0; i < n; i++)
    for (int j = i; j < i + 5; j++) count++;
```

[Show answer](#)

QUICK CHECK What is the Big O? (4)

JAVA

```
List<Integer> list = ... ;           // an ArrayList of size n
for (int x : queries)                // q queries
    if (list.contains(x)) found++;
```

[Show answer](#)

9. Analysing recursive algorithms — a first look

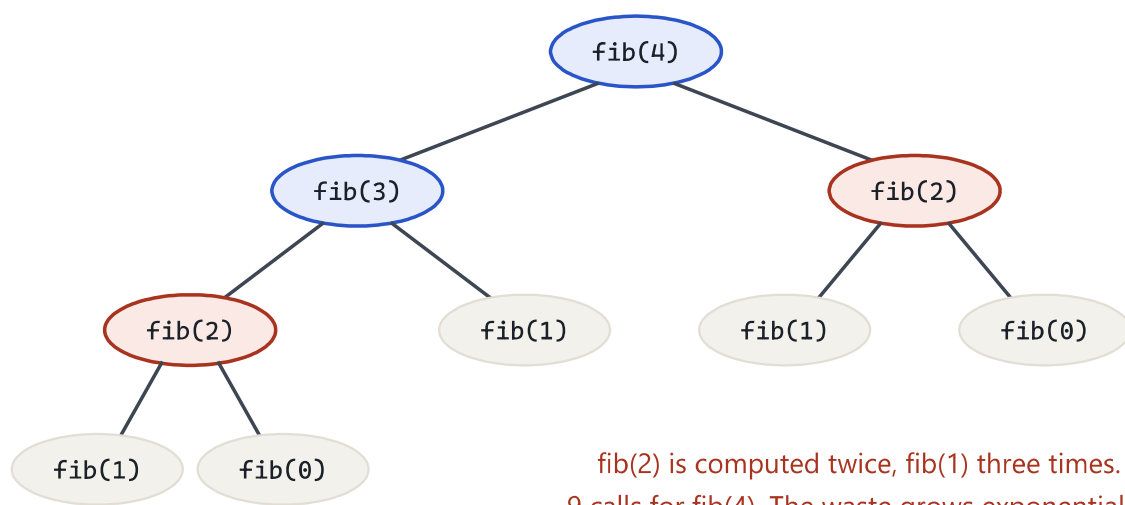
For recursion we write the cost as a **recurrence**: the cost for n in terms of the cost for a smaller input. We study recursion properly on Day 4. Today we only look at how the costs grow.

Algorithm	Recurrence	Solves to	Why
factorial(n)	$T(n) = T(n-1) + c$	$O(n)$	n calls, constant work each
binary search	$T(n) = T(n/2) + c$	$O(\log n)$	the problem halves each call
fib(n), naive	$T(n) = T(n-1) + T(n-2) + c$	$O(2^n)$	every call makes two more calls

JAVA RecursionCost.java

```
static long fib(int n) {
    calls++;
    if (n < 2) return n;           // base case
```

```
return fib(n - 1) + fib(n - 2);    // two recursive calls
}
```



The recursion tree of fib(4). Each level can double the number of calls, which is where 2^n comes from.

Output of `RecursionCost.java` :

n	fib(n)	calls made
10	55	177
20	6,765	21,891
30	8,32,040	26,92,537

Adding 10 to n multiplies the work by more than 100. On Day 8 we will bring this down to $O(n)$ with a hash table (**memoization**), and on Day 11 we will see that idea again as **dynamic programming**.

NOTE Recursion also costs space

Every call that has not finished yet sits on the **call stack**. factorial(n) needs n stack frames at its deepest point → **$O(n)$ space**, even though it does not create any array. Recursion that goes too deep gives Java's **StackOverflowError**.

10. Best, average and worst case

The same algorithm can take different time on different inputs of the **same size**. Linear search on 1,000 elements (`LinearSearchCount.java`):

Case	When it happens	Comparisons	Big O
Best	key is at index 0	1	$O(1)$
Average	key equally likely anywhere	about $n/2 = 500$	$O(n)$
Worst	key is last or not present	$n = 1,000$	$O(n)$

- **Worst case** is the promise we can always rely on. It is the one we usually quote.
- **Average case** tells us how the algorithm behaves normally. Quick sort is the famous example: $O(n \log n)$ on average, but $O(n^2)$ in its worst case (Day 7).
- **Best case** is rarely useful on its own. Almost any algorithm has a lucky input.

INSIDE JAVA.UTIL Amortised $O(1)$: `ArrayList.add`

Inside, an `ArrayList` is just an `Object[] elementData`. When the array is full, `add` creates a new array about **1.5× bigger** and copies everything across. That single `add` costs $O(n)$. But it happens so rarely that, averaged over many adds, each `add` costs **$O(1)$** . This is called **amortised** analysis. We will build this ourselves on Day 2.

11. Space complexity

DEFINITION Space complexity

The extra memory an algorithm needs, as a function of n . We usually count **auxiliary space**: memory beyond the input itself.

Code	Extra space
sum of an array with a loop	$O(1)$: a few variables

Code	Extra space
<code>onePass</code> second largest	$O(1)$
<code>bySorting</code> second largest (copies the array)	$O(n)$
recursive factorial(n)	$O(n)$: n stack frames
a 2-D <code>boolean[n][n]</code> "visited" table	$O(n^2)$

We often **trade space for time**. `HashSet` in Program B (Section 1) used $O(n)$ extra memory to save a huge amount of time. Deciding when that trade is worth it is a big part of this module.

INSIDE JAVA.UTIL

`int[]` vs `ArrayList<Integer>` — same Big O, very different memory

An `int` takes 4 bytes. An `Integer` is an **object**: an object header plus the value, plus a 4 or 8 byte reference to it stored in the list. On a typical 64-bit JVM, one million numbers take about 4 MB as `int[]` but roughly **4–5× more** as

`ArrayList<Integer>`. Big O hides constant factors. Real systems still care about them.

12. Big O traps in everyday Java

These are the mistakes that make "correct" Java programs slow in production. The timings are from one run of today's programs. Yours will differ, but the pattern will not.

Code that looks innocent	Hidden cost	Better choice
<code>s += x</code> inside a loop (String)	a new String is created and every old character copied on every <code>+=</code> → $O(n^2)$	<code>StringBuilder.append</code> → $O(n)$
<code>list.contains(x)</code> inside a loop	<code>ArrayList.contains</code> is a linear search → $O(n^2)$	<code>HashSet.contains</code> → $O(n)$ total
<code>for (i...)</code> <code>LinkedList.get(i)</code>	<code>get(i)</code> walks from one end of the list → $O(n^2)$	for-each loop or an <code>Iterator</code> → $O(n)$

Code that looks innocent	Hidden cost	Better choice
<code>ArrayList.remove(0)</code> in a loop	shifts every remaining element left → $O(n^2)$	<code>ArrayDeque.poll()</code> → $O(n)$ total
naive recursive <code>fib(n)</code>	$O(2^n)$ calls	memoization with a <code>HashMap</code> → $O(n)$ (Day 8)

Measured on one laptop:

Experiment	n	Slow version	Fast version
<code>StringConcatTrap</code>	80,000	541 ms	0 ms
<code>ProgramToInterface</code> (LinkedList <code>get(i)</code> loop)	80,000	2,618 ms	0 ms
<code>ListTraps</code> (<code>remove(0)</code> vs <code>poll()</code>)	2,00,000	1,777 ms	0 ms

WATCH OUT "My code works on 10 test records" is not enough

Every trap above runs instantly on small inputs. They only show up with real data volumes. That is exactly why we analyse **growth** and not just one test run.

13. Abstract Data Types — OO design for data

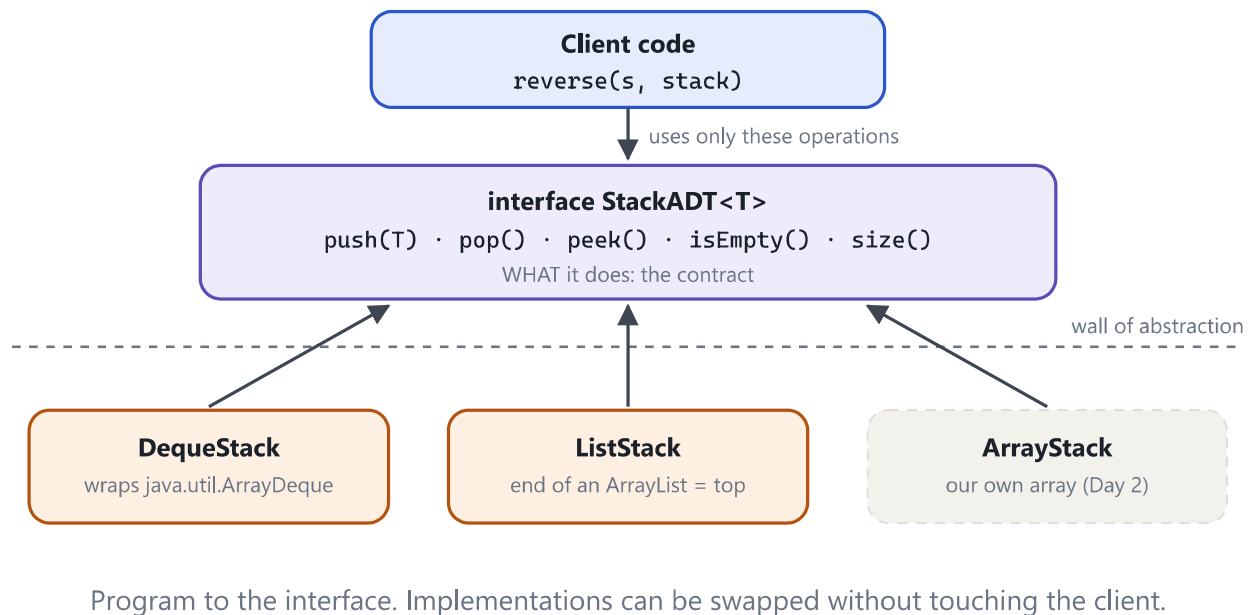
You already know **abstraction** and **encapsulation** from OOP. An ADT applies those same ideas to data.

DEFINITION Abstract Data Type (ADT)

A description of a type by **what operations it offers and how they behave**, with no mention of how it is stored. A **data structure** is one concrete way of storing the data to implement those operations.

Everyday example: an ATM offers *withdraw*, *deposit*, *check balance*. That is the ADT. How the bank stores your account — database, files, cloud — is the implementation. You use the ATM without knowing, and the bank can change its storage without retraining you.

	ADT	Data structure
Answers	What can I do?	How is it stored and done?
In Java	an <code>interface</code>	a <code>class</code> that implements it
Example	<code>List</code> , <code>Queue</code> , <code>Map</code> , our <code>StackADT</code>	<code>ArrayList</code> , <code>ArrayDeque</code> , <code>HashMap</code> , <code>ArrayStack</code>
Changes when...	the requirement changes	we need better speed or memory



Designing an ADT in Java

A good ADT states more than the method names. It is a **contract**:

1. **Operations** — names, parameters, return types.
2. **Behaviour** — what each operation does, including the special cases (what does `pop()` do on an empty stack?).
3. **Errors** — which exception is thrown, and when.
4. **Performance promise** — the expected Big O of each operation.
5. **Generics** — `StackADT<T>` , so it works for any element type.

JAVA `StackAdtDemo.java`

```

interface StackADT<T> {
    /** Adds item on top. Expected O(1). */
    void push(T item);

    /** Removes and returns the top item. Throws NoSuchElementException if empty
    T pop();

    /** Returns the top item without removing it. Throws NoSuchElementException
    T peek();

    boolean isEmpty();
    int size();
}

```

The client is written **once**, against the interface:

JAVA

```

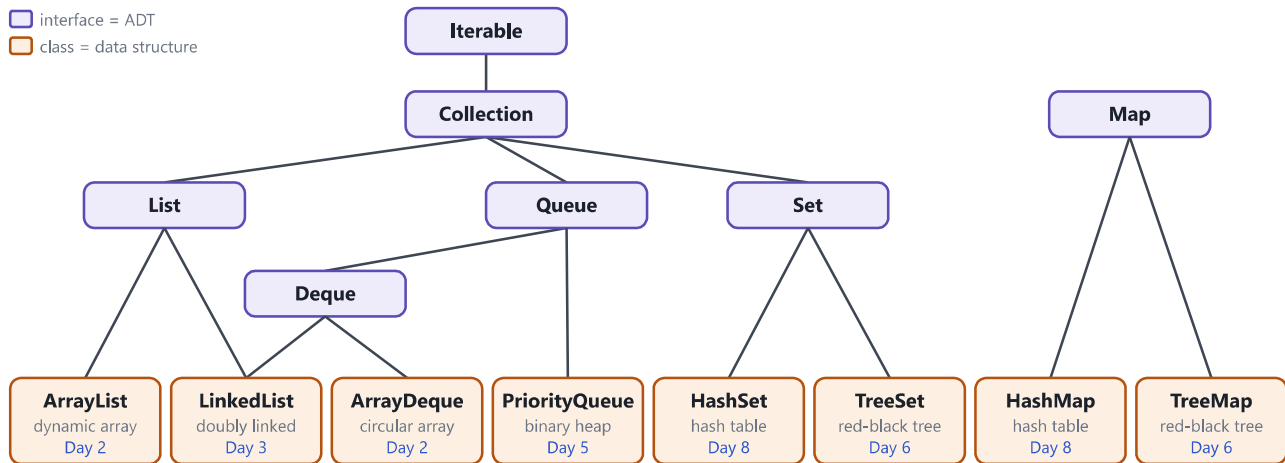
static String reverse(String s, StackADT<Character> stack) {
    for (char c : s.toCharArray()) stack.push(c);
    StringBuilder sb = new StringBuilder();
    while (!stack.isEmpty()) sb.append(stack.pop());
    return sb.toString();
}

reverse("ALGORITHM", new DequeStack<>()); // MHTIROGLA
reverse("ALGORITHM", new ListStack<>()); // MHTIROGLA: new implementation, sa

```

INSIDE JAVA.UTIL The Collections Framework is a library of ADTs

Every interface in `java.util` is an ADT, and every class is a data structure behind it. In this module we will **open each box**: build it ourselves, then see how the JDK does it.



The main parts of java.util: ADTs (interfaces) above, data structures (classes) below, and the day we open each one

"Program to the interface" is why the JDK can be designed like this:

JAVA

```
List<Integer> marks = new ArrayList<>(); // declared type = ADT, object = data
```

But the **same client code** can have **different complexity** depending on which implementation you pick (`ProgramToInterface.java`):

Loop over 80,000 items	ArrayList	LinkedList
<code>for (i...) list.get(i)</code>	4 ms	2,618 ms
for-each / <code>Iterator</code>	6 ms	0 ms

The ADT promises *what* happens. Only the data structure decides *how fast*. That is why a Java developer must know what is inside.

INSIDE JAVA.UUTIL The ADT contract inside Queue : two styles of the same operations

`java.util.Queue` offers every operation in two versions. One **throws an exception** on failure, the other **returns a special value**:

Operation	Throws exception	Returns special value
insert	<code>add(e)</code>	<code>offer(e)</code> returns <code>false</code>

Operation	Throws exception	Returns special value
remove	<code>remove()</code>	<code>poll()</code> returns <code>null</code>
examine	<code>element()</code>	<code>peek()</code> returns <code>null</code>

This is exactly the "errors" part of an ADT contract, designed deliberately.

WATCH OUT An OO design mistake inside the JDK: `java.util.Stack`

`Stack` (from Java 1.0) **extends** `Vector`, so it inherits list methods that break the stack ADT:

JAVA `JavaStackFlaw.java`

```
Stack<String> s = new Stack<>();
s.push("A"); s.push("B"); s.push("C");
s.add(0, "SNEAKY");           // insert at the BOTTOM of a stack?!
s.remove(2);                   // remove from the middle?!
```

Inheritance was used where **composition** was needed. Our `DequeStack` has an `ArrayDeque` inside it — it does not *extend* it. The `Stack` Javadoc itself recommends `Deque<Integer> stack = new ArrayDeque<>();` instead.

14. Summary — today on one page

- **Computational thinking:** decompose → recognise patterns → abstract → design an algorithm.
- **Our routine:** understand → examples and edge cases → brute force → improve → code → test and analyse.
- **Algorithm:** finite, definite, effective steps from input to output. It is built from **sequence, selection, iteration** (and recursion).
- **Analyse growth, not stopwatch time:** count operations as a function of n .
- **Big O rules:** drop constants and lower-order terms. Sequential blocks add, nested blocks multiply. Different inputs get different letters.

- **Growth ladder:** $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$.
- **Cases:** we quote the worst case. Amortised $O(1)$ = cheap on average across many operations (`ArrayList.add`).
- **Space:** auxiliary memory. Recursion uses stack space. We often trade space for time.
- **ADT = what (interface), data structure = how (class).** Program to the interface, but know the cost of the implementation.

15. Lab sheet — Day 1

Code for all tasks is in `code/day01/`. You can run any file directly with `java FileName.java` (Java 11+), or use your IDE.

LAB Task 1 — Complexity of loops (syllabus lab)

For each method in `LoopComplexity.java`, **first write your Big O on paper**, then run the program. For each column, check what happens when n doubles. Then add two loops of your own (for example `j = i; j < n; j++`, and `i = i * i` starting from 2) and predict their growth before running them.

LAB Task 2 — Complexity of recursive algorithms (syllabus lab)

Run `RecursionCost.java`. Add a recursive `power(x, n)` that multiplies x by itself n times, and count its calls. Then write `fastPower(x, n)` using $x^{2^k} = (x^k)^2$. Compare the call counts for $n = 1,000$. What is the Big O of each?

LAB Task 3 — Measure growth with `System.nanoTime()`

Run `GrowthExperiment.java`. Make a table with the ratio between rows (time for $2n \div$ time for n) for each column. Do the ratios come close to 2, a little over 2, and 4? Why are the first rows noisy? (Hint: JIT warm-up.)

LAB Task 4 — Same problem, better data structure

Run `DuplicateCheck.java`, `StringConcatTrap.java` and `ListTraps.java`. For each one, write in one line **why** the slow version is slow, using Big O.

LAB Task 5 — Design ADTs (preparation for Day 2)

Write an interface `QueueADT<T>` with `enqueue`, `dequeue`, `peek`, `isEmpty`, `size`. Add a Javadoc comment for each method stating its behaviour, its exception on an empty queue, and its expected Big O. Then write a class `ListQueue<T>` that implements it using an `ArrayList`. Which operation is slow in your implementation, and why? (We fix this on Day 2 with a circular queue.)

TRY IT Home challenge

1. Find the **k-th largest** element of an array. Give one $O(n \log n)$ solution and think about whether $O(n \times k)$ is better or worse.
2. Given an array of n integers, find whether any two of them add up to a target. Write the $O(n^2)$ version, then an $O(n)$ version using a `HashSet`.

16. Quick quiz

QUICK CHECK 1. Which grows fastest for large n ?

(a) $100n$ (b) $n \log n$ (c) $n^2/1000$ (d) $5000 \log n$

[Show answer](#)**QUICK CHECK 2. An algorithm takes 2 seconds for $n = 1,000$. It is $O(n^2)$. About how long for $n = 4,000$?**[Show answer](#)**QUICK CHECK 3. True or false: "An $O(n)$ algorithm is always faster than an $O(n^2)$ one."**[Show answer](#)**QUICK CHECK 4. What is the difference between `List` and `ArrayList`?**[Show answer](#)

**QUICK
CHECK**

5. Why is `Deque<Integer> st = new ArrayDeque<>()` preferred over `java.util.Stack`?

[Show answer](#)

Next class — Day 2: Arrays, Stacks & Queues. We build `ArrayStack`, a linear queue, a **circular queue** and a deque from scratch, and then open up `ArrayList` and `ArrayDeque` to see the same ideas inside the JDK.